Práctica 13 — Un enemigo que te persigue

Programación de Videojuegos · FP Lumara· Práctica previa: 12

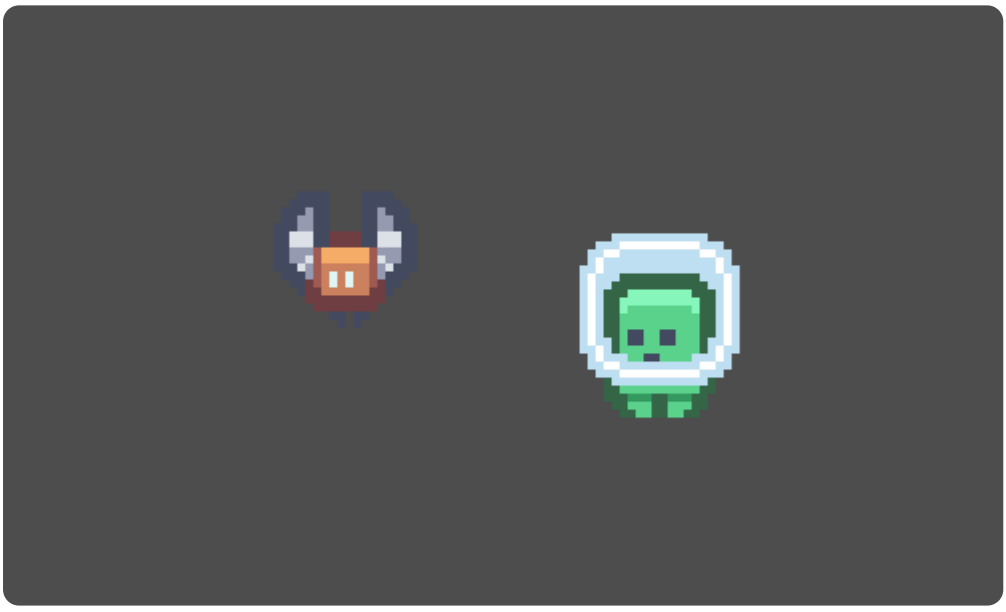
> *Lo que aprendes hoy: a que un nodo sepa quién es otro nodo, sin escribir rutas a mano.*

---

1. Qué vas a conseguir

Tu personaje de siempre, movido con el teclado, y un murciélago que **te persigue** por la pantalla. Cuando te alcanza, se queda a tu lado sin encimarse.

[CAPTURA 13.1]



2. Materiales

Descomprime `practica_13_inicio.zip` , que trae el personaje de la práctica 06 ya montado, más las tres imágenes del murciélago de la práctica 01.

---

3. La idea de hoy, en una frase

En la práctica 12 la nave perseguía **un punto** que le daba Godot: la posición del ratón.

Hoy el objetivo es **otro nodo**. El código es casi el mismo, pero aparece un problema nuevo: *¿cómo le digo al murciélago quién es el personaje?*

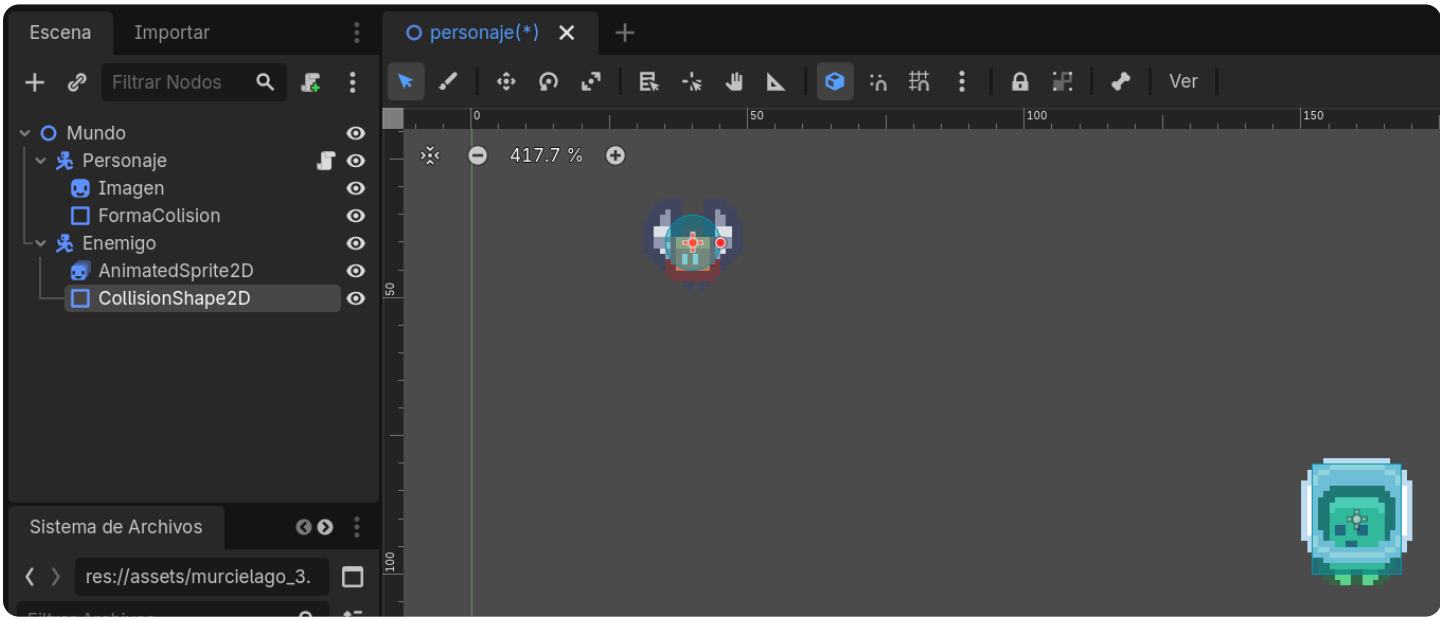
---

4. Paso a paso

4.1 Montar el murciélago

1. Abre `escenas/personaje.tscn` y guárdala como `escenas/mundo.tscn` con **Escena > Guardar Escena Como...** (así el nombre describe mejor lo que hay dentro).
2. Selecciona `Mundo` y añade un `CharacterBody2D` llamado `Enemigo`, en la posición `40, 40`.
3. Dentro de `Enemigo`, añade un `AnimatedSprite2D` llamado `Imagen` y móntale la animación `volar` con los tres PNG del murciélago, a 8 FPS y en bucle, como hiciste en la práctica 01. No olvides poner **Reproducción Automática** a `volar`.
4. Dentro de `Enemigo`, un `CollisionShape2D` llamado `FormaColision` con una **Nuevo CircleShape2D** de radio `5`.

[CAPTURA 13.2]



"Dos cuerpos montados igual: uno lo controlas tú, otro lo controlará el juego."

4.2 El script del enemigo

5. Con `Enemigo` seleccionado, añade un script en `res://scripts/enemigo.gd`:

```
extends CharacterBody2D

@export var objetivo: Node2D

var velocidad := 60.0
var distancia_minima := 20.0


func _physics_process(_delta: float) -> void:
    if objetivo == null:
        return

    var destino := objetivo.global_position

    if global_position.distance_to(destino) > distancia_minima:
        velocity = global_position.direction_to(destino) * velocidad
    else:
        velocity = Vector2.ZERO

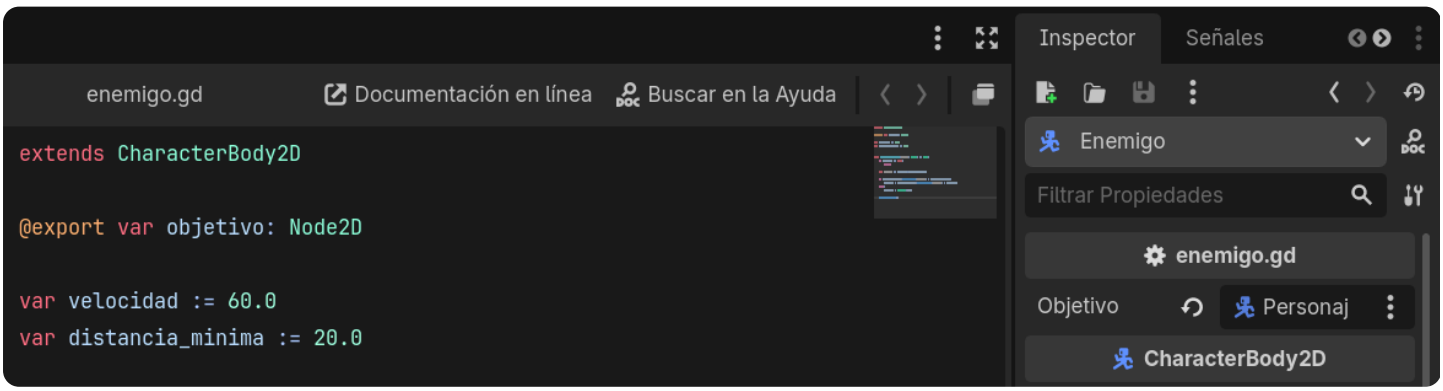
    move_and_slide()
```

6. Guarda el script. **Todavía no funciona**, y eso es lo interesante.

4.3 Decirle a quién perseguir

7. Selecciona el nodo `Enemigo` y mira el **Inspector**. Arriba del todo ha aparecido un campo nuevo llamado **Objetivo**, que no existía antes.
8. Ese campo lo ha creado la línea `@export`. Ahora **arrastra el nodo** `Personaje` desde el panel Escena hasta ese campo del Inspector.

[CAPTURA 13.3]



"@export crea una casilla en el Inspector. Ahí se arrastra el nodo."

9. Ejecuta con **F6** y huye del murciélago.

---

5. El código, línea a línea

`@export var objetivo: Node2D` La palabra `@export` hace que esta variable **aparezca en el Inspector**. Deja de ser un dato escondido en el código para convertirse en una casilla que se rellena arrastrando.

Es la primera vez en el curso que el código y el editor se hablan en esta dirección: hasta ahora el código leía cosas que tú habías puesto en el editor, y ahora es el código el que **crea** algo en el editor.

Por qué no usamos `$Personaje` como en la práctica 08

Podrías escribir `@onready var objetivo := $"../Personaje"` y funcionaría. Pero tiene dos problemas:

- La ruta depende de **dónde esté colocado** el enemigo en el árbol. Si lo mueves, se rompe.
- Si mañana quieres que ese mismo enemigo persiga a otra cosa, hay que tocar el código.

Con `@export`, el enemigo **no sabe ni le importa** a quién persigue: alguien se lo dice desde fuera. El mismo script vale para perseguir al personaje, a una moneda o a otro enemigo.

`if objetivo == null: return` `null` significa "aquí no hay nada". Si te olvidas de arrastrar el nodo en el Inspector, la variable está vacía y el juego reventaría al preguntarle su posición.

`return` significa "sal de la función ahora mismo, no hagas nada más". Así, si falta el objetivo, el enemigo simplemente se queda quieto en vez de romper el juego.

Es la primera **comprobación de seguridad** del curso, y es una costumbre que merece la pena coger.

El resto Es exactamente el mismo código de la práctica 12. Lo único que cambia es de dónde sale `destino`: antes era el ratón, ahora es la posición de otro nodo.

`distancia_minima` **es mayor hoy** 20 en vez de 8, porque aquí el objetivo es un personaje con cuerpo. Si el enemigo se acercara demasiado, los dos cuerpos chocarían y empezarían a empujarse. Con 20 se queda a una distancia prudente.

---

6. Pruébalo

- [ ] El personaje se mueve con el teclado como siempre.
- [ ] El murciélago se acerca sin que hagas nada.

- [ ] Cuando te alcanza, **se para** y no te empuja.
- [ ] Si huyes, vuelve a perseguirte.
- [ ] El murciélago aletea mientras vuela.
- [ ] En el panel **Salida** no hay nada en rojo.

---

7. La teoría en cinco líneas

- `@export` convierte una variable en una casilla del Inspector.
- Asignar nodos desde el Inspector es más robusto que escribir rutas en el código.
- Un script que no sabe a quién persigue **sirve para perseguir a cualquiera**.
- `null` significa que una variable está vacía; conviene comprobarlo antes de usarla.
- `return` sale de la función sin ejecutar lo que queda.

---

8. Si no te funciona

Lo que ves	Qué pasa	Cómo se arregla
El murciélago no se mueve	El campo Objetivo está vacío	Arrastra `Personaje` al Inspector
No aparece el campo Objetivo	No guardaste el script	Ctrl+S en el editor de código
Sigue sin aparecer	El script no está en `Enemigo`	Compruébalo
`Invalid access to property 'global_position' on a base object of type 'Nil'`	Falta la comprobación de `null` , y el objetivo está vacío	Las dos cosas
Se te sube encima y tiembla	`distancia_minima` demasiado pequeña	Súbela a 20
Te empuja por la pantalla	Los dos cuerpos se tocan	Mismo motivo, sube la distancia
Va tan rápido como tú y no se le escapa	`velocidad` igual o mayor que la tuya	El personaje va a 100; ponle 60
El murciélago está quieto y no aletea	Falta Reproducción Automática	Ponla a `volar`

---

9. Retos

Verde · Pon **tres murciélagos** con velocidades distintas (40, 60 y 80) y comprueba cómo se van separando. Cada uno necesita su propio objetivo asignado en el Inspector.

Amarillo · Haz que el murciélago **mire hacia donde va**, volteándose cuando te persigue desde la izquierda o desde la derecha. Ya lo hiciste con el personaje en el Hito 1: la propiedad es `flip_h` del `AnimatedSprite2D` .

Rojo · Que el murciélago solo te persiga si estás **a menos de 100 píxeles**, y que si te alejas más se quede quieto. Es lo que en los juegos se llama *rango de detección*. Después, piensa qué pasaría si además tuviera que volver a su sitio original al perderte, y escríbelo en tres líneas sin programarlo.

---

10. Autoevaluación

- [ ] Sé usar `@export` para crear una casilla en el Inspector.
- [ ] Sé asignar un nodo a esa casilla arrastrándolo.
- [ ] Sé explicar por qué es mejor que escribir la ruta en el código.
- [ ] Sé qué significa `null` y por qué conviene comprobarlo.
- [ ] Sé qué hace `return` .
- [ ] Entiendo por qué el código de persecución es el mismo que el de la práctica 12.

---

11. Vocabulario

Español	Inglés	Qué es
Exportar una variable	Export	Hacer que aparezca en el Inspector
Nulo	Null	Una variable que no apunta a nada
Rango de detección	Detection range	Distancia a la que un enemigo te ve
Comprobación de seguridad	Guard clause	Salir pronto si los datos no sirven

---

12. Entrega

Comprime la carpeta como `apellido_practica13.zip` y súbela al Aula Virtual de Alexia.

Capturas necesarias en esta práctica

Código	Contenido
13.1	Resultado final, a ser posible en GIF
13.2	Árbol completo con Personaje y Enemigo
13.3	Inspector del Enemigo con el campo Objetivo relleno